Quality Track

Interview Preparation Series • Professional Edition

Git & Dev Workflow

Compiled with LaTeX • June 17, 2026

harsha.somisetty01@gmail.com

In This Edition

> **How to use this file:** Read top-to-bottom for deep, mechanism-level understanding. Jump to §Common Interview Questions and §Revision Cheat Sheet for last-minute revision. Git fluency is *assumed* on the job — the differentiator in interviews is that you understand the **object model** underneath the commands, can reason about what `reset --hard` actually touches, and can recover from disasters calmly. Memorizing flags is worthless; understanding that "a branch is just a 41-byte file containing a hash" is everything.

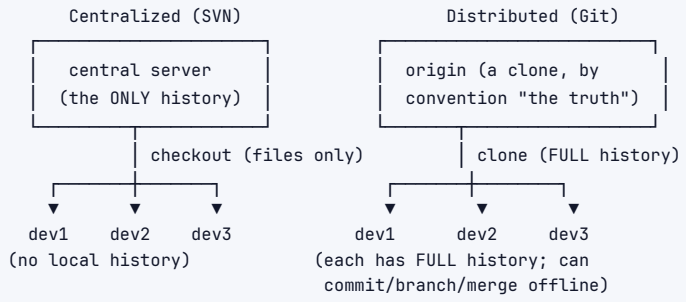
1 Overview --- What It Is

Git is a **distributed version control system (DVCS)**. Every clone is a *complete, independent copy* of the entire repository — all history, all branches, all tags — not just the latest snapshot. There is no privileged central server in Git's design; "origin" is just a convention. You can commit, branch, merge, view history, and time-travel entirely offline.

At its heart, Git is shockingly simple. Linus Torvalds famously described it as a **"content-addressable filesystem with a version-control UI bolted on top."** Strip away the porcelain commands (`add`, `commit`, `merge`) and you find a tiny **key-value store**: you give Git some content, it gives you back a hash (the key); you give it the hash later, it gives you back the exact content. Everything else — commits, branches, tags, history — is built from that one primitive.

Three things make Git fundamentally different from what came before:

1. **Snapshots, not diffs.** Most older systems (SVN, CVS, Perforce) store a file and then a *list of changes* to it over time. Git stores a **full snapshot** of the whole project at each commit (with deduplication so it's not wasteful). This makes branching and merging cheap and makes operations like `checkout` and `diff` between arbitrary points fast.
2. **Content addressing.** Every object (file content, directory tree, commit) is named by the **SHA-1 hash of its content**. Identical content → identical hash → stored once. This gives you free deduplication and tamper-evidence.
3. **Distributed.** Every clone is a backup. There is no single point of failure for *history*. The "server" (GitHub/GitLab) adds collaboration features (PRs, access control, CI) but Git itself doesn't require one.



2 Why It Exists

Coordinating many people editing the same codebase — without overwriting each other, while still being able to **undo, branch, review, audit, and bisect** — is impossible to do by hand (copying `project_final_FINAL_v2/` folders does not scale and has no merge story).

The problems version control solves:

1. **Concurrent editing.** Two people change the same file. Without VCS, one overwrites the other. Git lets both work in isolation and *merges* the changes, only stopping to ask a human when the edits truly conflict (touch the same lines).
2. **History & accountability.** Who changed this line, when, and *why*? `git blame` + the commit message answers this. Critical for debugging ("this broke in the deploy three weeks ago") and for institutional memory.
3. **Safe experimentation.** Cheap branches mean you can try a risky refactor, and if it fails, delete the branch — main is untouched.
4. **Reproducibility.** A commit hash pins the *exact* state of the codebase. CI builds, deploys, and rollbacks all reference immutable commits.
5. **Distributed resilience.** Every developer's laptop is a full backup. GitHub going down doesn't lose your history.

Why Git specifically (vs. earlier VCS):

Need	SVN/CVS (centralized)	Git (distributed)
Commit offline	No (needs server)	Yes
Branch creation cost	Expensive (server-side copy)	~Instant (write one file)
Merge quality	Painful, weak	Strong 3-way merge
Full history locally	No	Yes
Tamper-evidence	No	Yes (hash-chained)
Single point of failure	Yes (the server)	No (for history)

> Git stores **content addressed by hash**, so identical content is stored exactly once and history is tamper-evident: change any old commit and *every later hash changes*, instantly visible to everyone.

3 Why FAANG Cares

Git is the daily substrate of every engineer's work. Interviews rarely ask "what does `git add` do" — they probe whether you understand the *model* well enough to (a) keep history clean for reviewers, (b) recover from mistakes without panicking, and (c) reason about workflow at scale.

Google:

- Runs one of the largest **monorepos** on earth (`google3`), backed by Piper (a Perforce-derived system) and the Mondrian/Critique review tooling — but external repos and Android (AOSP, which uses Git + `repo`) are pure Git. Engineers are expected to produce small, reviewable, well-described CLs (changelists). The cultural value: *clean, atomic, bisectable history*. Knowing how `git bisect` finds a regression in O(log n) is directly relevant to debugging at Google scale.

Meta:

- Also a giant **monorepo** (backed by Mercurial-derived **Sapling/EdenFS** with `sl`, conceptually similar to Git). Trunk-based development with thousands of commits/day. They invest heavily in *virtual filesystems* and *sparse checkouts* because the repo is too big to fully materialize. Phabricator/Diffstack-style stacked diffs are core. Understanding why a monorepo needs sparse-checkout and VFS shows systems maturity.

Amazon:

- Heavy **polyrepo** culture (thousands of small service repos), tied to internal CI/CD (Pipelines, Brazil build system, CodeCommit). Two-pizza teams own their repos end-to-end. Interviews and on-the-job reviews value clean PRs, conventional commit hygiene, and the ability to cherry-pick hotfixes onto release branches.

Apple:

- Mixed; strong emphasis on commit signing, provenance, and careful release branching for OS versions. GPG/SSH-signed commits and tags matter for supply-chain trust.

Microsoft (GitHub, Azure DevOps):

- *Owns* GitHub. Migrated Windows (~300 GB, 3.5M files) to Git, which required inventing **VFS for Git** (now **Scalar**) and partial clone. The Windows-on-Git story is a famous case study in scaling Git. Azure DevOps and GitHub Actions are their CI/CD.

Netflix / Uber / Stripe / Databricks:

- Trunk-based development, feature flags, heavy CI gates, and "you build it, you run it." Clean Git workflow is part of operational safety — a botched force-push can take down a deploy pipeline.

The interview reality: When you say "I'd cherry-pick the fix onto the release branch," or "I'll rebase my feature branch to keep history linear, but I'll never rebase a shared branch," you're signaling that you understand collaboration safety and history hygiene — the things that separate a senior from a junior who only knows `git add . && git commit -m "stuff"`.

4 Core Concepts

4.1 Snapshots, Not Diffs (the mental-model shift)

This is the single most important conceptual leap. Imagine three commits where only one file changes each time:

```
SVN/CVS style (store deltas):
  C1: full A, full B, full C
  C2: Δ(A only)
  C3: Δ(B only)

  To reconstruct C3 you must
  replay C1 + all deltas.
```

```
Git style (store snapshots):
  C1: ┌snap┐ A1  B1  C1
  C2: ┌snap┐ A2  B1* C1*   (* = reuse, same hash)
  C3: ┌snap┐ A2* B2  C1*

To reconstruct C3, just read its
snapshot. Unchanged files point to
the SAME blob object (dedup).
```

In Git, a commit references a complete **tree** (the whole directory structure). Files that didn't change between commits **point to the exact same blob object** — same content, same hash, stored once. So you get the conceptual simplicity of "every commit is a full snapshot" with the storage efficiency of deltas (added later via packfiles). This is why `git checkout <old-commit>` is fast and why `git diff` between any two arbitrary commits is cheap.

4.2 Content Addressing

Every piece of data Git stores is named by the **hash of its content**, not by a filename or a sequential ID. The address *is* derived from the bytes. Consequences:

- **Deduplication is automatic.** Two files with identical bytes anywhere in history → one stored object.
- **Integrity is automatic.** If a byte on disk corrupts, the content no longer hashes to its name — Git detects it (`git fsck`).
- **History is tamper-evident.** Because a commit's hash includes its parent's hash (a hash chain, like a blockchain), altering any historical commit changes its hash, which changes its child's hash, cascading to every descendant. You cannot quietly rewrite the past.

4.3 A Branch Is Just a Pointer

The most liberating realization: **a branch is a 41-byte text file** (40 hex chars + newline) at `.git/refs/heads/<name>` containing one commit hash. Creating a branch writes one tiny file. Switching branches just moves `HEAD`. That's why branching in Git is instantaneous, unlike SVN where a branch was a server-side directory copy.

```bash
$ cat .git/refs/heads/main
3f8a1c9e2b...d4    # that's the entire branch. one hash.
$ cat .git/HEAD
ref: refs/heads/main   # HEAD points to the current branch
```

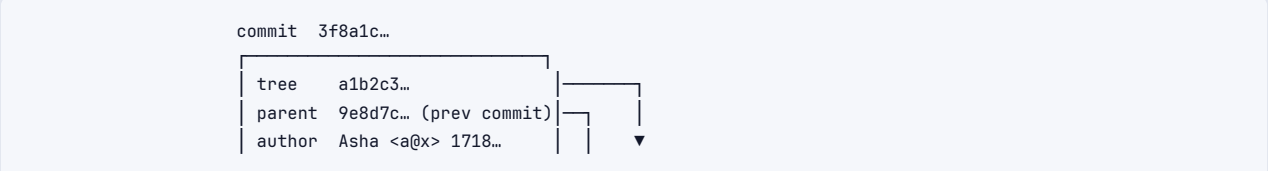
4.4 Commits Are Immutable; "Editing History" Means Rewriting

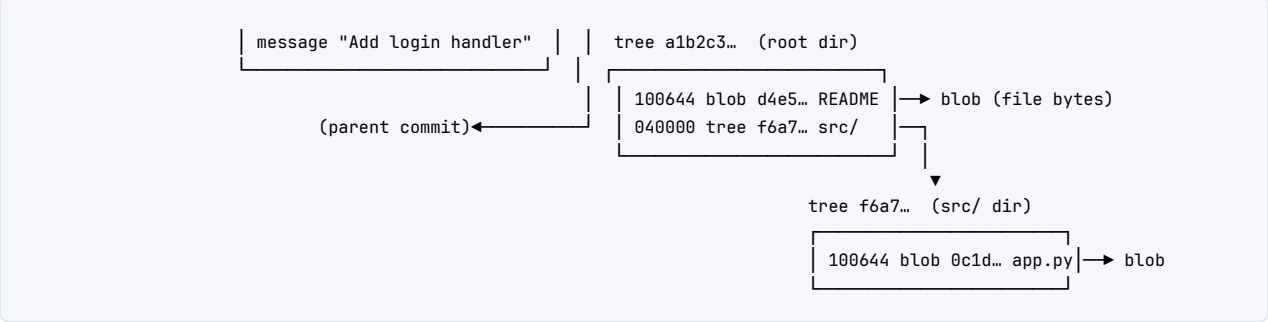
You never *modify* a commit. Operations that appear to edit history (`amend`, `rebase`, `commit --fixup`) actually **create brand-new commits with new hashes** and move a branch pointer to them. The old commits become unreferenced (dangling) and are eventually garbage-collected. This is why rebasing shared history is dangerous (covered later) — the old hashes others depend on vanish.

5 The Git Object Model

Everything in `.git/objects/` is one of **four object types**. Understanding these four is understanding Git.

Object	Stores	Named by	Points to
blob	file *contents* (no name, no mode)	SHA-1 of content	nothing
tree	a directory listing (names, modes, → blobs/trees)	SHA-1 of its entries	blobs and subtrees
commit	snapshot pointer + metadata	SHA-1 of commit text	one tree + parent commit(s)
tag (annotated)	a named, possibly signed pointer to an object	SHA-1 of tag text	usually a commit





- **blob** = the contents of a file. Note: a blob has *no filename*. The filename lives in the **tree** that references it. So renaming a file (same content) doesn't create a new blob — it changes a tree entry.
- **tree** = a directory. It maps `mode + name → blob/tree hash`. The root tree of a commit captures the whole project layout.
- **commit** = a tree (the snapshot) + zero-or-more **parents** + author/committer + timestamp + message. The first commit has zero parents; a normal commit has one; a merge commit has two or more.
- **tag** = a permanent named pointer. *Lightweight* tags are just refs; *annotated* tags are real objects (taggable, signable, with their own message).

5.1 How a Commit Hash Is Computed (worked)

A Git object's hash is the SHA-1 of: `"<type> <byte-length>\0" + content`. Let's compute a blob hash by hand-equivalent:

```bash
$ printf 'hello\n' | git hash-object --stdin
ce013625030ba8dba906f756967f9e9ca394464a

# Reproduce it manually: the header is "blob 6\0" (6 = len of "hello\n")
$ printf 'blob 6\0hello\n' | sha1sum
ce013625030ba8dba906f756967f9e9ca394464a  -    # identical!
```

A **commit** hash is the SHA-1 of the commit's text, which literally includes the tree hash, the parent hash, author/committer lines, and the message:

```bash
$ git cat-file -p HEAD
tree a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0
parent 9e8d7c6b5a4f3e2d1c0b9a8f7e6d5c4b3a2f1e0d
author  Asha Rao <asha@x.com> 1718000000 +0000
committer Asha Rao <asha@x.com> 1718000000 +0000

Add login handler
```

Because the **parent hash is part of the commit text**, changing any ancestor changes that commit's text → its hash → and therefore the hashes of *every* descendant. That cascade is the tamper-evidence property. It also means: **the same diff applied at a different point in history produces a different commit hash** (the parent differs). That single fact explains everything about rebase, cherry-pick, and "why rebasing shared history breaks people."

5.2 Identical Content Dedupes --- demonstrated

```Bash
$ echo "config" > a.txt
$ echo "config" > b.txt
$ git add a.txt b.txt
$ git ls-files -s
100644 5e40c0877058c504203932e5136051cf3cd3519b 0   a.txt
100644 5e40c0877058c504203932e5136051cf3cd3519b 0   b.txt
#              ^^^^ SAME hash — both filenames point to ONE blob object.
```

Two files, identical content → one blob stored. The tree records two *names* pointing at the same blob. This is automatic; you never ask for it.

5.3 `git cat-file` & `git hash-object` --- looking inside

```Bash
$ git cat-file -t 3f8a1c   # type of an object
commit
$ git cat-file -s 3f8a1c   # size in bytes
176
$ git cat-file -p a1b2c3   # pretty-print (here, a tree)
100644 blob d4e5f6…  README.md
040000 tree f6a7b8…  src

# Store arbitrary content and get its hash (without committing):
$ echo "test" | git hash-object -w --stdin
9daeafb9864cf43055ae93beb0afd6c7d144bfa4   # -w actually writes it to .git/objects
```

5.4 Packfiles & Delta Compression

Storing every snapshot as a full zlib-compressed blob (”loose objects”) is fine for a while, but Git periodically runs `git gc`, which packs loose objects into a **packfile** (`*.pack` + `*.idx`). *Within* a packfile, Git uses **delta compression**: similar objects (e.g., consecutive versions of a big file) are stored as one base + a chain of deltas. So Git's user-facing model is ”snapshots,” but its *storage* layer opportunistically uses deltas — best of both worlds. Note: Git chooses deltas by *content similarity heuristics* (size, filename), not by commit lineage; a delta base can even be a *newer* object.

```Bash
$ git gc                 # pack loose objects
$ git count-objects -vH  # see loose vs packed counts and sizes
$ git verify-pack -v .git/objects/pack/pack-*.idx | head   # see delta chains
```

5.5 Refs and HEAD

A **ref** is a human name → hash mapping, stored as a tiny file under `.git/refs/`:

```
.git/
├── HEAD                    → "ref: refs/heads/main"  (the current branch)
├── refs/
│   ├── heads/              local branches
│   │   ├── main            → 3f8a1c…
│   │   └── feature/login   → 7c4b2a…
│   ├── tags/               tags
```

```
│   │   └── v1.2.0            → e2f3a4…
│   └── remotes/origin/       remote-tracking branches (read-only mirrors)
│       └── main              → 3f8a1c…
```

- **HEAD** is "where am I?" Usually it's a *symbolic ref* pointing at a branch (`ref: refs/heads/main`). When you commit, Git appends a commit and moves the branch HEAD points to.
- **Detached HEAD** = HEAD points *directly* at a commit hash instead of a branch. New commits you make have no branch tracking them — easy to lose. (Recovery covered in Disasters.)
- **Remote-tracking refs** (`origin/main`) are local read-only snapshots of where the remote was at your last `fetch`. `git fetch` updates them; `git push` updates the remote and then your tracking ref.

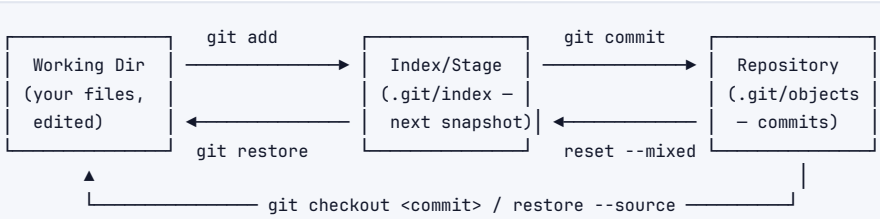
> **Interview takeaway:** "Git is a content-addressed key-value store of four object types (blob, tree, commit, tag); branches and HEAD are just movable pointers (refs) into that graph." If you can say that sentence and explain the commit-hash-includes-parent cascade, you've demonstrated you understand Git rather than memorized it.

5.6 SHA-1 → SHA-256

Git originally used **SHA-1** (160-bit). After the 2017 SHAttered SHA-1 collision, Git added two defenses: (1) a **collision-detection** variant of SHA-1 (`sha1dc`) that aborts on known-collision patterns, and (2) optional **SHA-256** repositories (`git init --object-format=sha256`). SHA-256 isn't yet the default because the ecosystem (hosting, tooling) is still migrating, and interoperability between SHA-1 and SHA-256 repos is a hard transition problem. For interviews: know that the hash is a content integrity mechanism, not a security boundary against a determined attacker — that's what *signing* is for.

6 The Three Areas & The Index

Git mediates between three "places," and the **index** (a.k.a. **staging area**) is the often-misunderstood middle layer that makes Git's commit model precise.



- **Working directory** — the actual files on disk you edit. This is the only place you see "uncommitted changes."
- **Index (staging area)** — a *binary file* `.git/index` that holds the **proposed next commit**: a flat list of `(mode, blob-hash, stage, path)` entries. It is, essentially, a pre-baked tree. `git add` writes the file's blob into the object store *and* records its hash in the index.
- **Repository** — `.git/objects/`, the permanent immutable history.

The index is what enables **partial / staged commits**: you can edit five files but stage only two, producing a focused commit. The classic confusing diagram from `git status` ("Changes to be committed" vs "Changes not staged for commit") is literally describing index-vs-working-dir differences.

6.1 Moving things between the areas

```bash
git add file.py              # working dir → index (stage)
git add -p                   # interactively stage SOME hunks of a file (see below)
git restore --staged file.py # index → working dir  (UNSTAGE; keeps your edits)
git restore file.py          # repo (HEAD) → working dir (DISCARD edits — destructive!)
git restore --source=HEAD~2 file.py   # pull an old version of a file into working dir
git rm --cached file.py      # stop tracking but keep the file on disk
git reset HEAD file.py       # older syntax for "unstage" (= restore --staged)
```

Modern Git (2.23+) split the overloaded `git checkout` into two clearer verbs:

- **`git switch`** — change branches.
- **`git restore`** — restore file contents (from index or a commit).

`checkout` still works for both but is ambiguous (it does branches *and* files), which historically caused accidental data loss when people typed `git checkout file` meaning to discard changes.

6.2 Partial staging with `git add -p`

`add -p` (patch mode) walks you through each *hunk* and asks whether to stage it:

```bash
$ git add -p app.py
@@ -10,7 +10,9 @@ def handler():
-    return None
+    log.info("handling")      # the bug fix you want NOW
+    return process(req)
@@ -40,3 +42,6 @@ def debug():
+    breakpoint()              # a stray debug line you DON'T want to commit
Stage this hunk [y,n,q,a,d,s,e,?]?
```

You answer `y` to the fix hunk and `n` to the `breakpoint()` hunk → your commit contains only the fix. This is a *hygiene* superpower: it lets you craft clean, single-purpose commits even when your working directory is messy. `s` splits a hunk; `e` lets you hand-edit which lines stage.

> **Interview takeaway:** The index exists so a commit is an *intentional, curated snapshot*, not "whatever happens to be on disk." Mentioning `git add -p` to keep commits atomic signals strong hygiene.

7 Branching & Merging

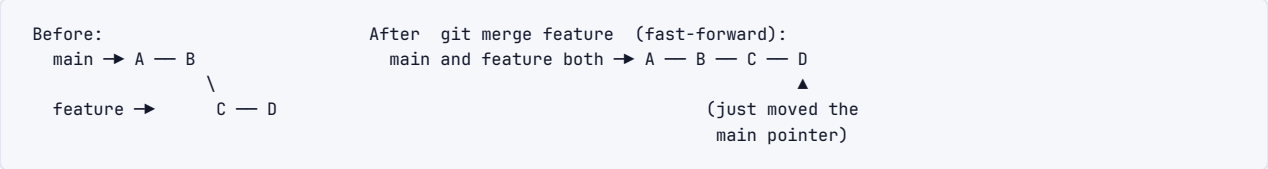
7.1 A branch is a ref; here's what creating/switching does

```bash
git branch feature/login        # write .git/refs/heads/feature/login = <current HEAD hash>
git switch feature/login        # point HEAD at that branch (update working dir to match)
git switch -c feature/login     # create AND switch in one step
```

Creating a branch is O(1): write one 41-byte file. Switching updates HEAD and reconciles your working tree to the target commit's snapshot.

7.2 Fast-forward merge (no divergence)

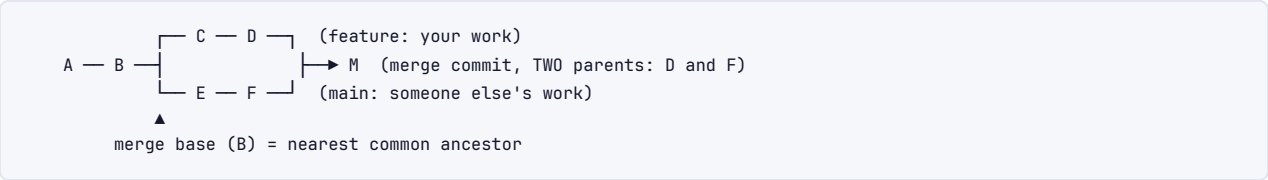
If the branch you're merging *in* is strictly ahead of your current branch — your branch is an ancestor of it — Git can just **slide the pointer forward**. No merge commit, no new content.



`main` had no commits of its own that `feature` lacked, so merging is just `main = feature`. Use `git merge --ff-only` to *require* a fast-forward (fail loudly if a real merge would be needed) — common in CI to keep history linear. Use `--no-ff` to *force* a merge commit even when a fast-forward is possible (preserves the "this was a feature branch" topology).

7.3 Three-way merge (divergent branches)

When both branches have new commits since they split, Git performs a **3-way merge** using the **common ancestor** (merge base) plus the two tips:



The algorithm (default strategy **ort**, formerly **recursive**):

1. Find the **merge base** B (`git merge-base main feature`).
2. For each file, compare three versions: **base (B)**, **ours (main)**, **theirs (feature)**.
3. If only one side changed a region → take that change automatically.
4. If both sides changed the *same* region differently → **conflict**; ask the human.
5. Produce a new **merge commit M** with *two parents* (D and F), recording that both lines of history merged here.

Why "3-way" beats "2-way": a plain 2-way diff between two files can't tell *which* side made a change — it only sees they differ. The base gives Git the reference point to say "ours added this line; theirs is unchanged here, so keep ours" vs "both edited this same line → conflict."

The **recursive/ort** strategy handles the case where two branches have *multiple* common ancestors (criss-cross merges): it recursively merges the ancestors into a single virtual base. **ort** ("Ostensibly Recursive's Twin") is a faster, more correct rewrite that became the default in Git 2.34 — better rename detection, fewer spurious conflicts, much faster on big repos.

7.4 Merge conflicts: markers and resolution

When both sides edit the same lines, Git writes both versions into the file with markers and pauses:

```python
<<<<<<< HEAD (ours — current branch)
timeout = 30
=======
timeout = 60
>>>>>>> feature/tune-timeouts (theirs — incoming)
```

```bash
git status                # lists "both modified" files
# edit the file: pick one, combine them, delete ALL the <<<< ==== >>>> markers
git add config.py         # mark THIS file resolved
git merge --continue      # (or git commit) once all conflicts resolved
git merge --abort         # bail out entirely, restore pre-merge state

# Shortcuts when you know which whole side you want, per file:
git checkout --ours   config.py && git add config.py   # keep current branch's version
git checkout --theirs config.py && git add config.py   # keep incoming version
```

Enable **`diff3`/`zdiff3` conflict style** to *also* show the merge base — it makes resolution far easier because you can see what the original was:

```bash
$ git config --global merge.conflictstyle zdiff3
<<<<<<< HEAD
timeout = 30
||||||| base
timeout = 45         # ← the original; now you can see WHO changed WHAT
=======
timeout = 60
>>>>>>> feature
```

Turn on **`rerere`** ("reuse recorded resolution") so Git remembers how you resolved a given conflict and auto-applies it next time the same conflict reappears (huge for long-lived branches and repeated rebases):

```bash
git config --global rerere.enabled true
```

7.5 Octopus merge (3+ branches at once)

`git merge a b c` creates a single commit with *more than two* parents — an **octopus merge**. It only works when there are no conflicts (it refuses if manual resolution is needed). Mostly used to bundle independent topic branches; rarely seen day-to-day, but a good "do you know edge cases?" answer.

> **Interview takeaway:** Be able to draw the merge-base diagram and explain *why* the third reference point (the base) is what makes auto-merging possible. Then explain that a conflict is simply "both sides changed the same region relative to the base, so Git refuses to guess."

8 Rebase vs Merge

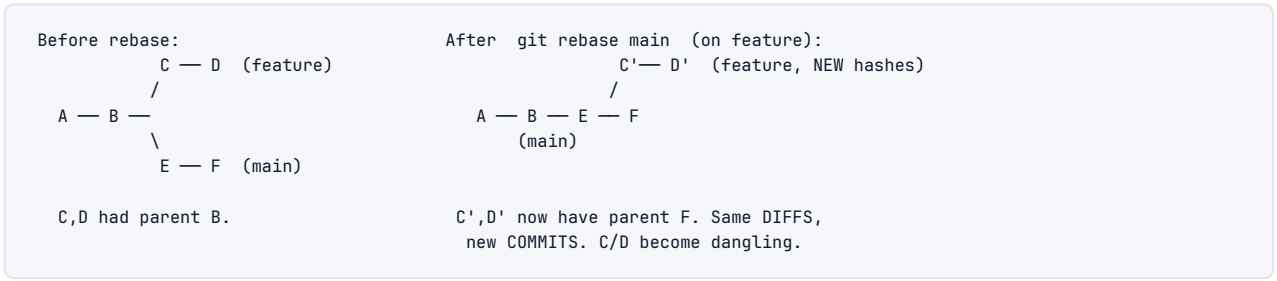
Both integrate changes from one branch into another. The difference is *topology* and *whether new commits are created*.

8.1 Merge = preserve history, add a merge commit

Merge keeps the true, branching history and ties it together with a merge commit (two parents). Nothing is rewritten; all original commits keep their hashes.

8.2 Rebase = replay your commits onto a new base (new hashes!)

`git rebase main` takes *your* commits, sets them aside, fast-forwards your branch to `main`, then **re-applies each of your commits one at a time** on top — producing **brand-new commits with new hashes** (different parents → different commit text → different SHA). The result is a clean linear history *as if* you'd started your work from the latest main.




```bash
git switch feature
git rebase main          # replay feature's commits on top of main
# conflicts? resolve, then:
git add <file> && git rebase --continue
git rebase --abort       # back out, restore original state
git rebase --skip        # drop the current conflicting commit and continue
```

8.3 Side-by-side

Aspect	Merge	Rebase
History shape	Branching, with merge commits	Linear, flat
Rewrites commits?	No (original hashes kept)	**Yes** (new hashes)
Traceability of ”when branches met”	Preserved (merge commit)	Lost (looks sequential)
Conflict resolution	Once, at the merge	Possibly once *per replayed commit*
Safe on shared/pushed branches?	Yes	**No** (see Golden Rule)
`git bisect` / `git log` readability	Noisier graph	Cleaner, easier to bisect

8.4 Interactive rebase --- the history-editing superpower

`git rebase -i` opens an editor listing your commits with an action per line. This is how you craft a clean PR out of messy work-in-progress commits.

```bash
$ git rebase -i HEAD~5
pick   a1b2c3 Add login form
squash 4d5e6f WIP fix typo            # fold into previous, combine messages
fixup  7g8h9i oops forgot import      # fold into previous, DISCARD this message
reword 0j1k2l Add validatoin          # keep commit, edit its message (fix typo)
edit   3m4n5o Refactor auth service   # stop here so you can amend/split
drop   6p7q8r debug print statement   # delete this commit entirely
# you can also REORDER lines to reorder commits
```

Action	Effect
`pick`	keep the commit as-is
`reword`	keep the commit, edit its message
`edit`	pause at this commit to amend it or split it into several
`squash` (`s`)	merge into the previous commit, *combine* both messages
`fixup` (`f`)	merge into the previous commit, *discard* this message
`drop` (`d`)	remove the commit entirely
(reorder lines)	reorders the commits

Worked example — turning 5 messy commits into 2 clean ones:

```
WIP commits in your branch:              After interactive rebase:
  a1b2  Add login form                     X1  Add login form + validation
  4d5e  WIP fix typo           —squash→        (3 commits folded into one,
  7g8h  oops forgot import     —fixup—→         message cleaned)
  0j1k  Add validatoin         —reword→
  6p7q  debug print statement  —drop——→    (gone)
```

You end up with two logically distinct, well-described commits a reviewer will thank you for.

8.5 Autosquash --- fixups without manual reordering

Make a fix and tag it to a specific earlier commit; later, autosquash sorts and folds it automatically:

```bash
git commit --fixup=a1b2c3        # creates "fixup! Add login form"
# ...more work...
git rebase -i --autosquash main  # Git auto-positions the fixup under a1b2c3 and marks it 'fixup'
```

Set `git config --global rebase.autosquash true` to make `--autosquash` the default.

8.6 The GOLDEN RULE: never rebase commits that have been pushed/shared

Rule: *Only rebase commits that live solely in your local repo and that nobody else has based work on.* Never rebase a branch others have pulled (like `main`, `develop`, or a shared feature branch).

Exactly why it breaks collaborators: rebasing replaces commits `C, D` with new commits `C', D'` (different hashes). Suppose you already pushed `C, D` and a teammate pulled them and built `G` on top of `D`. Now you rebase, force-push `C', D'`, and:

```
Remote now has:  A — B — E — F — C'— D'
Teammate has:    A — B ———————— C — D — G   (built on the OLD D)
```

When your teammate pulls, Git sees their `C, D` as *unrelated* to your `C', D'` (different hashes). Their history and yours have **diverged**. They get a messy merge, duplicate commits, and conflicts — and if they force-push to "fix" it, they may clobber *your* `C', D'`. The old commits they depended on effectively vanished from the shared branch. Multiply across a team and you get "the repo is broken" Slack threads. The fix-up dance is painful (`git rebase --onto`), which is why the rule is *don't*.

8.7 `git pull --rebase`

Default `git pull` = `fetch` + **merge**, which sprinkles tiny "Merge branch 'main' of origin" commits all over history. `git pull --rebase` = `fetch` + **rebase** your local commits on top of the fetched remote tip → linear history, no noise. Safe here because your *local-only* commits are the ones being replayed (not shared ones). Many teams set it as default:

Bash

```bash
git config --global pull.rebase true
# or per-pull:
git pull --rebase origin main
```

> **Interview takeaway:** "Rebase rewrites commits into new hashes for a linear history; merge preserves topology with a merge commit. Rebase locally for cleanliness, merge to integrate shared branches, and *never* rebase anything already pushed and depended upon." That one sentence is the senior-level answer.

9 Undo, Reset & Recovery

The thing that trips everyone up: `reset`, `revert`, `restore`, and `checkout` all "undo," but touch *different areas* with *different safety profiles*.

9.1 `git reset --soft / --mixed / --hard` --- what each touches

`reset` moves the current **branch pointer** to a target commit and *optionally* updates the index and working dir. The mode controls how far the changes "fall back":

Mode	Moves branch HEAD	Resets **Index**	Resets **Working Dir**	Result / use
`--soft`				Changes from undone commits become **staged**. Use to re-commit differently / combine last N commits.
`--mixed` (default)				Changes become **unstaged** edits in working dir. Use to unstage / redo the commit.
`--hard`				**Everything wiped** to target. *Destructive* — uncommitted work is gone.

```
                ┌─ working dir
        ┌─ index ┤
HEAD ───┤        └─ working dir
--soft : move HEAD only          → diffs sit in INDEX (staged)
--mixed: move HEAD + index       → diffs sit in WORKING DIR (unstaged)
--hard : move HEAD + index + WD  → diffs DELETED
```

Bash

```bash
git reset --soft HEAD~3   # "undo last 3 commits but keep all changes staged" (squash-by-hand)
git reset --mixed HEAD~1  # "undo last commit, keep changes as edits to re-stage"
git reset --hard HEAD~1   # "throw away last commit AND its changes" — danger
git reset --hard origin/main  # "make my branch exactly match the remote" — danger
```

> `--hard` is the only common Git command that destroys *uncommitted* working-tree changes irrecoverably (they were never in a commit, so reflog can't help). Committed work it removes is still recoverable via reflog (below).

9.2 `git revert` --- the safe, public undo

`revert` does **not** rewrite history. It creates a **new commit** that applies the *inverse* of a target commit's diff. History moves *forward*; the bad change is neutralized without removing anything. This is the **only** correct way to undo a commit on a **shared/public** branch.

```
Before:  A ── B ── C(bad) ── D
After  git revert C:
         A ── B ── C(bad) ── D ── C⁻¹   (new commit that undoes C; C still in history)
```

Bash

```bash
git revert <bad-sha>          # make an inverse commit (opens editor for message)
git revert HEAD               # undo the most recent commit, safely
git revert -m 1 <merge-sha>   # revert a MERGE commit (-m 1 = keep first parent's line)
git revert --no-commit A B C  # stage inverses of several commits into one revert
```

9.3 `reset` vs `revert` --- when each is safe

	`git reset`	`git revert`
Mechanism	Moves branch pointer back (rewrites tip)	Adds an inverse commit (moves forward)
Rewrites history?	Yes	No
Safe on shared branch?	**No** (forces force-push)	**Yes**
Use when	Local, unpushed cleanup	Undoing something already pushed

9.4 `restore` / `checkout` --- file-level

```bash
git restore file.py                # discard unstaged edits to file (from index/HEAD) — destructive
git restore --staged file.py       # unstage (index → keep working edits)
git restore --source=HEAD~2 app.py  # bring an OLD version of a file into working dir
git checkout <commit> -- file.py   # older equivalent of restore --source
```

9.5 `git clean` --- remove untracked files

Git won't auto-delete *untracked* files; `clean` does, and it's destructive (no reflog for files never tracked). **Always dry-run first.**

```bash
git clean -n          # DRY RUN: show what WOULD be deleted
git clean -fd         # actually delete untracked files (-f) and dirs (-d)
git clean -fdx        # also delete ignored files (build artifacts, node_modules) — be careful
```

9.6 Safety summary

```
Recoverable via reflog (commits were made):  amend, reset (any), rebase, branch delete
NOT recoverable (never committed):           reset --hard on uncommitted edits,
                                             git clean, git restore <file> (unstaged edits),
                                             git stash drop (after the fact)
```

> **Interview takeaway:** "`reset` rewinds the branch pointer (and optionally index/working dir) — local use only; `revert` adds an inverse commit and is the safe way to undo something already pushed. `--hard` and `clean` are the two commands that can lose *uncommitted* work for good."

10 Powerful Commands (bisect, cherry-pick, reflog, stash, worktree)

10.1 `git reflog` --- your undo history / safety net

The **reflog** records *every* movement of HEAD and branch tips locally (commits, resets, rebases, checkouts, merges) — even ones that left commits "dangling" and unreachable from any branch. It's how you recover from almost any "I lost my work" disaster. Reflog entries are local, not pushed, and expire after ~90 days (30 for unreachable).

```bash
$ git reflog
3f8a1c9 HEAD@{0}: reset: moving to HEAD~3
a1b2c3d HEAD@{1}: commit: Add payment retry
7c4b2ae HEAD@{2}: commit: Add login handler
e2f3a4b HEAD@{3}: checkout: moving from main to feature
```

Scenario A — undo a bad `reset --hard`: You ran `git reset --hard HEAD~3` and panic. The three commits are unreachable but not gone yet.

```bash
git reflog                      # find the hash BEFORE the reset (e.g. a1b2c3d HEAD@{1})
git reset --hard a1b2c3d        # restore the branch to that exact state
```

Scenario B — recover a deleted branch:

```bash
git branch -D feature/login     # oops, deleted unmerged branch
git reflog                      # find the last commit that WAS feature/login's tip
git switch -c feature/login <that-sha>   # recreate the branch at that commit
```

Scenario C — undo a botched rebase: Before any rebase, `HEAD@{n}` records the pre-rebase tip.

```bash
git reflog                      # find "rebase (start): ..." — the commit just before it is your old tip
git reset --hard HEAD@{5}       # or the specific pre-rebase hash
# Even simpler if it was the last operation:
git reset --hard ORIG_HEAD      # ORIG_HEAD = where HEAD was before the last "big" move
```

10.2 `git bisect` --- binary-search a regression in O(log n)

A test passes at an old commit and fails now. Somewhere in the (say) 1,024 commits between, one broke it. Linear search = up to 1,024 checks. Bisect = `log2(1024)` = **~10 checks** because each step halves the suspect range.

```bash
git bisect start
git bisect bad                  # current commit is broken
git bisect good v1.4.0          # this old tag was fine
# Git checks out the MIDPOINT commit. You test it, then tell Git:
git bisect good                 #   ...if this midpoint works
git bisect bad                  #   ...if it's broken
# repeat ~log2(N) times; Git narrows to the FIRST bad commit
git bisect reset                # done — return to your original HEAD
```

Automate it with a script that exits 0 (good) / non-zero (bad):

```bash
git bisect start HEAD v1.4.0    # bad=HEAD, good=v1.4.0 in one line
git bisect run pytest tests/test_checkout.py::test_total   # Git runs it at each step automatically
# Use exit code 125 in your script to say "skip — can't test this commit"
```

The output `<sha> is the first bad commit` plus that commit's diff usually points straight at the bug. Bisect is the single biggest argument for keeping commits **small and each-one-building** (so you can bisect cleanly).

10.3 `git cherry-pick` --- copy a commit onto another branch

Applies the *diff* of a specific commit as a **new commit** (new hash) on your current branch. The canonical use: a bug is fixed on `main`, and you need that exact fix on the `release/2.3` branch without pulling in everything else.

```bash
git switch release/2.3
git cherry-pick a1b2c3d              # apply that fix here as a new commit
git cherry-pick a1b2c3d^..f6e7d8     # a RANGE of commits
git cherry-pick -x a1b2c3d           # append "(cherry picked from commit a1b2c3d)" to the message
# conflict? resolve, then:
git add <file> && git cherry-pick --continue
git cherry-pick --abort
```

Caveat: cherry-picking the *same* change onto multiple branches creates *duplicate* commits (different hashes, same diff). When those branches later merge, Git is usually smart enough to detect the patch is already present, but it can cause spurious conflicts. Prefer cherry-pick for genuine hotfix backports, not as a substitute for merging.

10.4 `git stash` --- shelve work-in-progress

Saves your uncommitted changes (working dir + staged) onto a stack and reverts to a clean tree, so you can switch context (e.g., urgent hotfix) without committing half-done work.

```bash
git stash                       # shelve tracked changes, clean the working dir
git stash push -m "wip: auth"   # named stash
git stash -u                    # also stash UNtracked files
git stash list                  # stash@{0}: WIP on feature: ...
git stash show -p stash@{0}     # view the diff
git stash pop                   # re-apply the latest stash AND remove it from the stack
git stash apply stash@{1}       # re-apply a specific stash, KEEP it on the stack
git stash branch fix stash@{0}  # create a branch from a stash (if it won't apply cleanly)
git stash drop stash@{0}        # delete one stash
git stash clear                 # nuke all stashes (no easy recovery)
```

Stashes are real commits under the hood (`refs/stash`), so a *dropped* stash is recoverable via `git fsck --unreachable | grep commit` for a while — but it's fragile; don't rely on it.

10.5 `git worktree` --- multiple working dirs, one repo

Normally one repo = one working directory on one branch. `worktree` lets you check out **multiple branches simultaneously** in separate folders, all sharing the same `.git` object store (no second clone, no duplicated history). Great for: building a release branch while developing on a feature, or reviewing a PR without stashing.

```bash
git worktree add ../hotfix release/2.3   # new folder ../hotfix checked out to release/2.3
git worktree add -b experiment ../exp    # create branch 'experiment' in ../exp
git worktree list
git worktree remove ../hotfix
```

10.6 `git blame` --- who/when/why for each line

```bash
git blame app.py                # annotate every line with commit/author/date
git blame -L 40,60 app.py       # only lines 40-60
git blame -w -C app.py          # ignore whitespace (-w), detect moved/copied code (-C)
git log -S "functionName"       # "pickaxe": find commits that ADDED/REMOVED that string
git log -L :handler:app.py      # follow the history of a single function
```

`blame` tells you the commit; the commit message tells you *why* — which is the whole point of writing good commit messages.

10.7 `git log` tricks

```bash
git log --oneline --graph --all --decorate   # the classic visual history graph
git log --since="2 weeks ago" --author="Asha"
git log main..feature            # commits on feature NOT yet on main (what your PR adds)
git log feature..main            # commits on main that feature is missing
git log -p app.py                # full diffs touching app.py
git shortlog -sn                 # commit counts per author
git log --first-parent           # follow only mainline (skip merged-in branch detail)
```

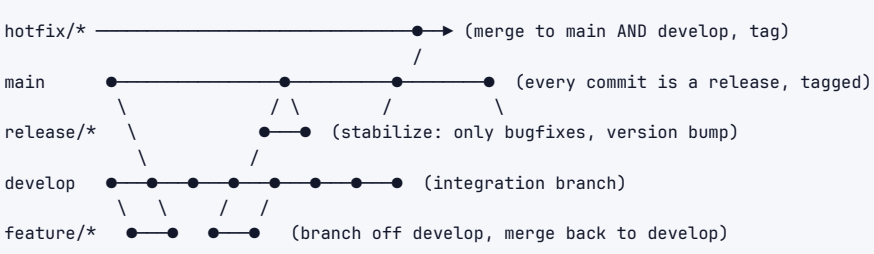
> **Interview takeaway:** Name-drop the *right* tool for the *right* problem: "regression → `bisect`; lost commit → `reflog`; backport a fix → `cherry-pick`; context-switch → `stash`; parallel branches → `worktree`." That fluency reads as senior.

11 Branching Workflows

A *workflow* is the team agreement about how branches map to releases. The right choice depends on release cadence, team size, and CI maturity.

11.1 Gitflow (heavyweight, release-train teams)

Two long-lived branches (`main` = production, `develop` = integration) plus short-lived `feature/*`, `release/*`, and `hotfix/*` branches.



- **feature/** → branch from `develop`, merge back to `develop`.
- **release/** → branch from `develop` to stabilize a version; only bugfixes; merge to *both* `main` and `develop`; tag on `main`.
- **hotfix/** → branch from `main` for emergency prod fixes; merge to *both* `main` and `develop`.

Good for: versioned software with scheduled releases (installed apps, libraries, mobile with app-store review). Downsides: many long-lived branches, painful merges, slow integration

("merge hell") — overkill for continuously-deployed web services.

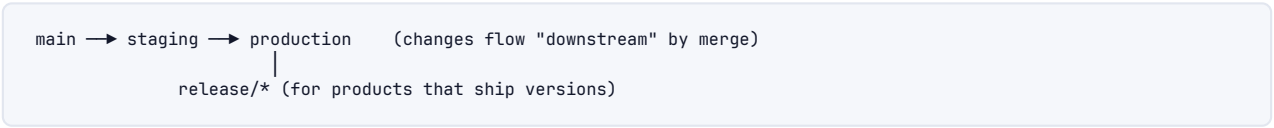
11.2 GitHub Flow (lightweight, continuous deploy)

One long-lived branch (`main`, always deployable). Everything else is a short-lived feature branch → PR → review → CI → merge → deploy.

Good for: web apps with continuous deployment, small/medium teams. Simple, fast. Assumes strong CI and feature flags for unfinished work.

11.3 GitLab Flow (GitHub Flow + environment/release branches)

Adds environment branches (`staging`, `production`) or release branches to GitHub Flow, so code promotes via merges: `main → staging → production`. Bridges "always deploy" simplicity with the reality of staged rollouts and regulated releases.



11.4 Trunk-Based Development (TBD) --- what big-tech actually does at scale

Everyone commits to a single **trunk** (`main`) many times a day, in tiny increments, behind **feature flags**. Branches are either nonexistent or live hours-to-a-day. CI runs on every commit; merges are continuous, so there's no "big bang" integration.

- **Feature flags** decouple *deploy* from *release*: incomplete code ships to prod dark (flag off) and is toggled on when ready. This is why you can integrate constantly without shipping broken features.
- **Why it scales:** continuous small merges = trivial conflicts (you're always near HEAD), no long-lived divergence, no "merge hell." It's the only model that works at Google/Meta scale with thousands of engineers.
- Requires: fast, reliable CI; feature flags; good test coverage; sometimes release branches cut *from* trunk for stabilization.

11.5 Comparison

Workflo	Long-lived branches	Branch lifetime	Integration freq	Best for	Main risk
Gitflow	main + develop (+release/hotfix)	days–weeks	low	versioned/installed software, scheduled releases	merge hell, complexity
GitHub Flow	main	hours–days	high	continuous-deploy web apps	needs strong CI
GitLab Flow	main + env/release	hours–days	high	staged rollouts, compliance	more branches to manage

Workflo	Long-lived branches	Branch lifetime	Integration freq	Best for	Main risk
Trunk-Based	main only	hours	continuous	large teams, high deploy velocity	needs feature flags + great CI

> **Interview takeaway:** "Small teams shipping continuously → GitHub Flow / trunk-based with feature flags. Versioned products with release trains → Gitflow. At FAANG scale, trunk-based with flags is the norm because continuous tiny merges avoid long-lived divergence and merge hell." Tie the choice to *release cadence and CI maturity*, not personal taste.

12 Monorepo vs Polyrepo

Monorepo: all projects/services in *one* repository (one history, one CI config, one set of tooling).
Polyrepo: each project/service in its *own* repository.

```
Monorepo:                          Polyrepo:
  one-big-repo/                      repo: payments-service
    services/payments/               repo: auth-service
    services/auth/                   repo: web-frontend
    libs/common/                     repo: shared-lib  (versioned & published)
    web/frontend/                    ...each with its own CI, owners, releases
```

Dimension	Monorepo	Polyrepo
Cross-project change	*Atomic* — one commit changes a lib + all its callers, one PR, one review	Multi-repo dance: version-bump the lib, publish, update each consumer separately
Shared tooling/CI	One config, consistent everywhere	Each repo configures its own (drift)
Dependency mgmt	"Live at HEAD" — everyone on the same version, no version hell	Semantic versioning, registries, diamond-dependency conflicts
Code visibility / reuse	Everything discoverable; easy refactors across boundaries	Siloed; reuse via published packages
Build/scale	Needs special tooling (Bazel, Buck) to not rebuild the world; clone gets huge	Each repo small and fast to clone/build
Blast radius	A bad commit/CI break can affect everyone	Naturally isolated per repo
Access control	Coarser (whole repo) — needs path-based ACLs	Fine-grained per repo

Why monorepos need heavy tooling: a naive `git clone` of Google/Meta's repo is impractical (hundreds of GB, millions of files). They rely on:

- **Sparse-checkout / partial clone** — materialize only the directories you work in.
- **Virtual filesystems** — Meta's **EdenFS**, Microsoft's **VFS for Git / Scalar** — files appear present but are fetched on access.
- **Build systems** — **Bazel** (Google), **Buck** (Meta) — content-addressed, hermetic, incremental builds with remote caching so you don't "rebuild the world."

Real examples:

- **Google:** a single monorepo (`google3`, billions of LOC) on Piper, "live at HEAD," one build system (Blaze/Bazel). Atomic cross-cutting refactors via large-scale change tooling.
- **Meta:** monorepo on Sapling/EdenFS; sparse virtual checkouts; trunk-based.
- **Microsoft:** Windows in Git via VFS for Git/Scalar (the famous 300 GB / 3.5M-file migration).
- **Amazon:** the *opposite* — thousands of small service polyrepos, matching two-pizza team autonomy and independent deploy cadence.

> **Interview takeaway:** The trade-off is **atomic cross-project changes + uniform tooling (monorepo)** vs **isolation + independent scaling + simpler per-repo ops (polyrepo)**. Monorepos buy consistency at the cost of needing serious build/VCS tooling; polyrepos buy isolation at the cost of dependency/version coordination. The "right" answer is "it depends on org structure (Conway's Law) and tooling investment."

13 Hooks, Submodules, Subtrees, LFS, Sparse-Checkout, Signing

13.1 Hooks

Scripts Git runs automatically at lifecycle points, stored in `.git/hooks/` (local, not committed) — teams share them via tools like **pre-commit** or **Husky**.

Hook	Fires	Common use
`pre-commit`	before a commit is created	lint, format, run fast tests, block secrets
`commit-msg`	after message entered	enforce conventional-commit format
`pre-push`	before `push`	run test suite, block pushing WIP/`--fixup!`
`post-merge` / `post-checkout`	after merge/checkout	re-install deps, rebuild
server-side `pre-receive`	on the server before accepting a push	enforce policy (signed commits, no force-push to main)

Because local hooks aren't committed and can be skipped (`--no-verify`), real enforcement lives in **CI** (server-side). Local hooks are for fast feedback; CI is the gate.

13.2 Submodules vs Subtrees

Both embed one repo inside another, differently:

- **Submodule** — a *pointer* (a recorded commit hash) to an external repo, checked out into a subdirectory. The parent repo stores only the reference, not the code. Pros: clear separation, the child has its own history. Cons: notoriously fiddly — `git clone --recursive`, `git submodule update --init`, detached HEADs, easy to forget to commit the pointer bump.

```bash
git submodule add https://github.com/org/lib vendor/lib
git submodule update --init --recursive
```

- **Subtree** — the external repo's *files and history are merged into* a subdirectory of the parent (no extra metadata for cloners). Pros: cloners need nothing special; the code is just there. Cons: pulling upstream updates uses special subtree commands; history is intertwined.

```bash
git subtree add --prefix=vendor/lib https://github.com/org/lib main --squash
git subtree pull --prefix=vendor/lib https://github.com/org/lib main --squash
```

13.3 Git LFS (Large File Storage)

Git is terrible at large binaries (every version is stored whole; they don't delta well and bloat every clone forever). **LFS** replaces big files (videos, models, PSDs) with tiny text *pointers* in the repo and stores the actual bytes on a separate LFS server, fetched on checkout.

```bash
git lfs install
git lfs track "*.psd" "*.bin"    # writes patterns to .gitattributes
git add .gitattributes
```

13.4 Sparse-checkout

Materialize only part of a (large/mono) repo's tree into your working dir, while history still references everything.

```bash
git clone --filter=blob:none --sparse <url>   # partial clone: skip blobs until needed
git sparse-checkout set services/payments libs/common
```

13.5 Signing (provenance / supply-chain trust)

Hashes prove *integrity* (content wasn't corrupted), not *authenticity* (who made it). **Signed commits/tags** prove authorship.

```bash
git config commit.gpgsign true
git commit -S -m "Release 2.4.0"          # GPG-sign
git tag -s v2.4.0 -m "v2.4.0"             # signed annotated tag
git log --show-signature
# Modern alternative: SSH-key signing (no GPG needed)
git config gpg.format ssh
git config user.signingkey ~/.ssh/id_ed25519.pub
```

GitHub shows a ”Verified” badge for signed commits — increasingly required for release tags and supply-chain security (e.g., Sigstore/gitsign).

14 Disasters & Recovery (Worked Scenarios)

Stay calm; almost everything committed is recoverable via **reflog**. Here are the seven situations interviewers love.

14.1 1) I committed to `main` instead of my feature branch

Your last 2 commits should have been on a branch. `main` hasn't been pushed yet.

```bash
git branch feature/login            # create feature branch pointing at current (correct) state
git reset --hard origin/main        # rewind LOCAL main back to the remote (drops the 2 commits FROM
↪  main)
git switch feature/login            # your commits live safely here now
```

If you'd already pushed those commits to `main` on a shared remote, **do not** reset — `cherry-pick` them onto the branch and `revert` them on main instead (see #2).

14.2 2) Undo a commit already pushed to a shared branch

History is shared — **never** `reset`/force-push. Use `revert` (forward-moving inverse commit):

```bash
git revert <bad-sha>      # creates an inverse commit
git push origin main      # safe, fast-forward push; no rewriting, no clobbering teammates
# Reverting a merge commit:
git revert -m 1 <merge-sha>   # -m 1 keeps the mainline parent
```

14.3 3) Accidental `reset --hard`, lost work

The commits are unreachable but still in the object store (until GC).

```bash
git reflog                        # find the hash from BEFORE the reset, e.g. a1b2c3d HEAD@{1}
git reset --hard a1b2c3d          # restore exactly
# Or, if you only need it on a new branch:
git switch -c recovered a1b2c3d
```

Caveat: if `--hard` wiped *uncommitted* edits (never committed), reflog can't help — those bytes were never an object.

14.4 4) Force-pushed and clobbered a teammate's commits

You `git push --force` and overwrote commits the teammate had pushed. The remote's old tip is gone *server-side*, but it likely still exists in *someone's* local reflog or `origin/…` tracking ref.

```bash
# Teammate (or anyone who fetched the old state) finds the old tip:
git reflog show origin/main          # or check their local branch reflog
git push --force-with-lease origin <old-good-sha>:main   # restore
```

Prevention: always use **`git push --force-with-lease`** instead of `--force`. `--force-with-lease` refuses the push if the remote moved since your last fetch (i.e., someone else pushed) — it *won't* silently clobber. And protect `main`/release branches server-side (GitHub branch protection: no force-push, require PRs).

14.5 5) Committed a secret / huge file --- purge from ALL history

Removing it in a new commit is **not enough** — it's still in history (and a leaked secret must be considered compromised → **rotate the credential immediately**, then scrub). Rewriting history requires `git filter-repo` (recommended) or **BFG**:

```bash
# Preferred: git-filter-repo (faster, safer than the deprecated filter-branch)
git filter-repo --path config/secrets.yml --invert-paths   # strip a file from ALL history
git filter-repo --strip-blobs-bigger-than 50M              # strip oversized blobs

# Alternative: BFG Repo-Cleaner (simpler for the common cases)
bfg --delete-files secrets.yml
bfg --strip-blobs-bigger-than 50M
git reflog expire --expire=now --all && git gc --prune=now --aggressive
```

This **rewrites every commit hash** → everyone must re-clone (a coordinated, disruptive operation). Then **force-push** (with team coordination) and **invalidate the leaked secret**. Order of operations: rotate the secret first, *then* scrub history, *then* notify the team to re-clone.

14.6 6) Detached HEAD --- I made commits and now they're "gone"

You `git checkout <sha>` (detached), committed, then switched away. Those commits aren't on any branch.

```bash
# Right after committing in detached state, BEFORE switching:
git switch -c rescue        # attach a branch to current HEAD → commits saved

# If you ALREADY switched away:
git reflog                  # find the dangling commit you made
git switch -c rescue <that-sha>
```

A detached HEAD itself is harmless (Git warns you); the danger is only *losing track* of commits you make in it.

14.7 7) A gnarly conflict during a big merge/rebase

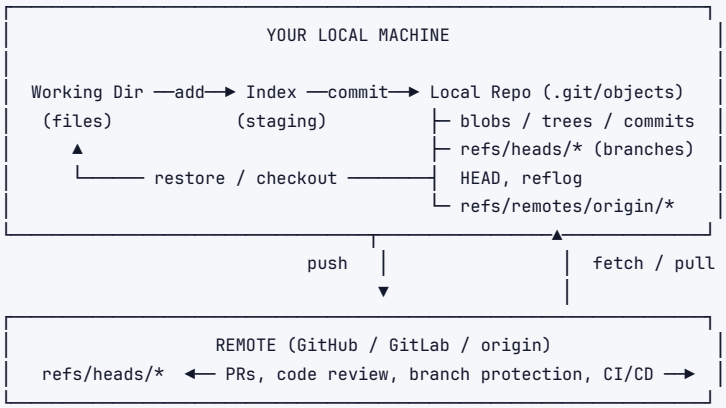
```bash
git config merge.conflictstyle zdiff3   # show the base too — see who changed what
git config rerere.enabled true          # remember resolutions for repeats
git status                              # list conflicted files
git mergetool                           # launch a 3-way visual tool (vimdiff, meld, vscode...)
# Resolve each file; remove ALL <<<< |||| ==== >>>> markers; then:
git add <file>
git merge --continue      # or: git rebase --continue
# Escape hatches:
git merge --abort
git rebase --abort
# Per-file "just take one side":
git checkout --ours  config.py && git add config.py
git checkout --theirs config.py && git add config.py
```

For a *huge* conflicted rebase, consider `git rebase --onto` to replay only the commits you need, or split it into smaller steps. If a conflict recurs every rebase, `rerere` saves your sanity.

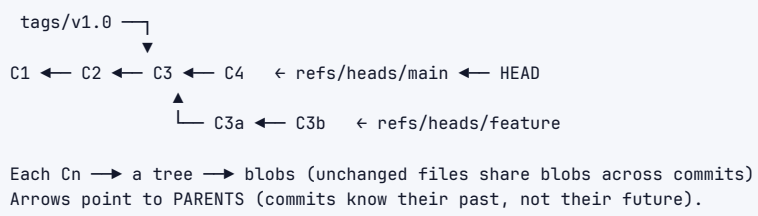
> **Interview takeaway:** The meta-answer to "you broke the repo, what now?" is: **don't panic, `git reflog` first, prefer non-destructive fixes (`revert`, `--force-with-lease`), and never rewrite shared history without team coordination.** That composure is exactly what they're testing.

15 Architecture / Diagrams

15.1 The full data flow



15.2 Object graph for a small history



15.3 Merge vs Rebase topology (recap)

16 Real-World Examples

- **Hotfix backport (cherry-pick):** A null-pointer crash is fixed on `main` (commit `a1b2c3`). Customers on the `release/3.2` line need it now. `git switch release/3.2 && git cherry-pick a1b2c3` → tag `v3.2.1` → ship. The full `main` (with unrelated risky features) never touches the release branch.
- **Finding a perf regression (bisect):** A dashboard got 4× slower ”sometime last sprint.” `git bisect start; git bisect bad HEAD; git bisect good v2.7.0; git bisect run ./bench.sh` walks ~9 commits and prints the offending commit — a seemingly innocent change to a query that dropped an index hint.
- **Cleaning a PR (interactive rebase):** 14 ”WIP”, ”fix”, ”oops” commits become 3 logical commits via `git rebase -i` (squash/fixup/reword) before requesting review. Reviewers see intent, not your keystroke history.
- **Leaked AWS key (filter-repo):** An engineer pushes `.env` with a live key. Response: rotate the key (assume compromised), `git filter-repo --path .env --invert-paths`, force-push with team coordination, everyone re-clones, add `.env` to `.gitignore` and a `pre-commit` secret scanner.

- **Microsoft Windows on Git:** ~300 GB, 3.5M files, 4,000+ engineers — impossible with vanilla Git; solved by inventing VFS for Git (now Scalar) with on-demand file hydration and partial clone.
- **Google "live at HEAD":** one monorepo, everyone builds against trunk; a library change and all its callers update in a *single atomic commit* — impossible in a polyrepo without coordinated multi-repo releases.

17 Real-Life Analogies

- **Commits = save points in a video game.** Each is a full snapshot you can reload. The hash is the save-slot ID; the parent link is "the save you made before this one."
- **Branches = bookmarks / sticky notes** on the timeline. Moving a branch = peeling the sticky note off one commit and putting it on another. Cheap because it's just a sticky note, not a copy of the book.
- **Staging area = a shopping cart.** Your working dir is the whole store (all your changes). You put *selected* items in the cart (`git add`), then check out (`git commit`). `git add -p` is picking individual items off a shelf rather than sweeping the whole shelf in.
- **Merge = combining two edited copies of a document** by hand, with the *original* as reference (3-way) so you know who changed what. A conflict is "you both rewrote the same paragraph differently."
- **Rebase = re-recording your podcast episodes on top of the newest intro** — same content, brand-new recordings (new hashes). Don't re-record episodes listeners already downloaded (the Golden Rule).
- **Reflog = your browser history for HEAD.** Even after you "close the tab" (delete a branch / hard reset), the history remembers where you were, so you can go back.
- **Cherry-pick = copying one paragraph from one document into another**, rather than merging the whole document.
- `revert` **= an erratum/correction note** appended at the end ("the change on page 5 was wrong, here's the fix") vs `reset` **= ripping the page out** as if it never existed.
- **Content addressing = a coat check that gives you a ticket equal to a fingerprint of the coat** — identical coats get identical tickets, and you can't swap a coat without the ticket changing.

18 Memory Tricks / Mnemonics

- **Four objects: "BTCT" — Blob, Tree, Commit, Tag.** (Blob = bytes, Tree = directory, Commit = snapshot+parent, Tag = name.)
- **Three areas: "Work, Stage, Store"** (working dir → index → repo). Verbs: **add** moves right, **restore** moves left.
- **Reset modes — "Soft keeps Staged, Mixed keeps Modified, Hard kills it":**
  - **S**oft → index (**S**taged).
  - **M**ixed → working dir (**M**odified/unstaged).
  - **H**ard → gone.
- **Reset vs Revert: "Reset Rewinds (private), Revert Records (public)."**
- **Golden Rule: "Never rebase what you've pushed."** (Or: *"Rebase local, merge shared."*)
- **Pull default: "pull = fetch + merge; add** `--rebase` **for a clean line."**
- **Disaster reflex: "Reflog first, panic never."**
- **Pick the tool: "Regression→Bisect, Lost→Reflog, Backport→Cherry-pick, Pause→Stash."**
- `--force-with-lease`**, not** `--force`**:** "lease checks the lock before you smash it."

19 Common Interview Questions

19.1 Q1: Does Git store diffs or snapshots? Why does it matter?

Model answer: Git stores **full snapshots** per commit, with unchanged files pointing to the same blob (so it's not wasteful). Conceptually each commit is a complete tree; physically, packfiles add delta compression. This makes branch/checkout/diff between *any* two commits fast and uniform, and it's why merging is cheaper than in delta-based systems where you'd replay a chain of changes. Most older systems (SVN/CVS) store a base plus per-file deltas, so reconstructing an old state means replaying changes.

Follow-ups:

- *How does it avoid storing duplicate content?* → Content addressing: identical bytes hash to the same blob, stored once; packfiles further delta-compress similar objects.
- *Then what are packfiles?* → A storage optimization (`git gc`) that packs loose objects and delta-compresses similar ones; the logical model stays "snapshots."

19.2 Q2: Walk me through the four Git object types and how a commit hash is formed.

Model answer: **Blob** = file contents (no name). **Tree** = a directory listing mapping names+modes to blobs/subtrees. **Commit** = a pointer to one root tree + parent commit(s) + author/committer/message. **Tag** = a named (optionally signed) pointer. A commit's hash is `SHA-1("commit <len>\0" + commit-text)`, and the commit text *includes the tree hash and the parent hash*. Because the parent's hash is baked in, changing any ancestor changes that commit's hash and cascades to all descendants — that's the tamper-evidence and why the same diff at a different parent yields a different commit.

Follow-ups:

- *Where does a filename live?* → In the tree entry, not the blob — so a pure rename reuses the blob.
- *SHA-1 is broken — is Git insecure?* → Git uses collision-detecting SHA-1 and supports SHA-256 repos; the hash is for integrity/dedup, not an authenticity guarantee — that's what signing is for.

19.3 Q3: Explain rebase vs merge and when you'd use each.

Model answer: Merge integrates by creating a **merge commit** with two parents — it preserves true history and never rewrites commits. Rebase **replays** your commits onto a new base, creating *new commits with new hashes* for a **linear** history. I rebase my *local* feature branch to keep it clean and easy to review/bisect, and to incorporate the latest `main` without a merge bubble. I **merge** to integrate shared branches and to preserve "these commits were one feature." The hard rule: never rebase commits already pushed/shared, because rewriting their hashes breaks anyone who built on the originals.

Follow-ups:

- *Why exactly does rebasing shared history break people?* → It replaces `C,D` with `C',D'`; teammates who based work on `C,D` now diverge — duplicate commits, messy merges, and a force-push can clobber work.
- *`git pull` default?* → fetch+merge; I prefer `pull --rebase` to avoid noise merge commits on local-only work.

19.4 Q4: `reset --soft` vs `--mixed` vs `--hard`?

Model answer: All move the branch pointer to a target commit; they differ in how far they reset. `--soft` moves HEAD only, leaving the undone changes **staged** (good for re-doing/combining

the last N commits). `--mixed` (default) also resets the index, leaving changes as **unstaged** edits. `--hard` also resets the working dir — **destroying** uncommitted changes. `--soft`/`--mixed` are safe; `--hard` can lose uncommitted work permanently (reflog recovers commits, not never-committed edits).

Follow-ups:

- *Undo the last 3 commits but keep all the code to recommit as one?* → `git reset --soft HEAD~3` then commit.
- *You hard-reset and lost commits — recover?* → `git reflog` → `git reset --hard <pre-reset-sha>`.

19.5 Q5: How do you undo a commit that's already been pushed to a shared branch?

Model answer: Use `git revert <sha>`, which creates a *new* commit applying the inverse diff, then push normally. It moves history forward and doesn't rewrite shared commits, so no one is disrupted and no force-push is needed. `reset` + force-push would rewrite shared history and break collaborators. For a merge commit, `git revert -m 1 <merge-sha>` to keep the mainline parent.

Follow-ups:

- *Later you want the change back?* → Revert the revert, or cherry-pick the original.
- *When is `reset` acceptable then?* → Only on local, unpushed commits nobody depends on.

19.6 Q6: A bug appeared "sometime in the last 200 commits." How do you find it fast?

Model answer: `git bisect` — a binary search over the commit range. Mark a known-good commit and the current bad one; Git checks out the midpoint, I test it and mark good/bad, and it halves the range each step — ~8 tests for 200 commits instead of 200. I'd automate it with `git bisect run <test-script>` (exit 0 = good, non-zero = bad, 125 = skip) so Git finds the first bad commit unattended. The output is the exact offending commit and its diff.

Follow-ups:

- *What makes bisect work well?* → Small, individually-building commits; if a commit doesn't build, mark it `skip` (exit 125).
- *Complexity?* → O(log n) tests.

19.7 Q7: How do you take a single fix from `main` onto a release branch?

Model answer: `git cherry-pick <fix-sha>` while on the release branch — it applies just that commit's diff as a new commit, without dragging in unrelated changes. I'd use `-x` to record the original SHA in the message for traceability, then tag a patch release. It's the standard backport mechanism. Caveat: it creates a duplicate commit (different hash), so I avoid using cherry-pick as a substitute for actually merging long-running branches.

Follow-ups:

- *Range of commits?* → `git cherry-pick A^..B`.
- *Conflict during pick?* → resolve, `git add`, `git cherry-pick --continue` (or `--abort`).

19.8 Q8: Explain the 3-way merge and why conflicts happen.

Model answer: Git finds the **merge base** (nearest common ancestor) and compares three versions of each file: base, ours, theirs. If only one side changed a region, Git takes that change automatically; if *both* sides changed the *same* region relative to the base, Git can't safely choose, so it marks a **conflict** with `<<<< ==== >>>>` markers and asks me to resolve. The base is what makes auto-merge possible — a plain two-way diff can't tell which side made a change. The default strategy is **ort** (a faster, more correct successor to recursive), which handles multiple common ancestors by recursively merging them into one virtual base.

Follow-ups:

- *Make conflicts easier?* → `merge.conflictstyle=zdiff3` (shows the base) and `rerere.enabled` (reuse past resolutions).
- *Take one whole side per file?* → `git checkout --ours/--theirs <file>` then `git add`.

19.9 Q9: You committed a secret (API key) and pushed it. What now?

Model answer: First, **rotate the credential immediately** — once pushed, treat it as compromised; scrubbing alone is not enough. Then remove it from *all* history with `git filter-repo --path <file> --invert-paths` (or BFG), expire reflogs and GC, and force-push with team coordination since this rewrites every subsequent hash and everyone must re-clone. Finally, add the file to `.gitignore` and add a `pre-commit`/CI secret scanner so it can't recur. Order matters: rotate → scrub → re-clone.

Follow-ups:

- *Why isn't a "delete the file" commit enough?* → The secret still lives in history and in the remote; anyone can check out the old commit.
- *filter-repo vs filter-branch?* → filter-branch is slow and deprecated/error-prone; filter-repo (or BFG) is the recommended tool.

19.10 Q10: What does "a branch is just a pointer" mean, and what's a detached HEAD?

Model answer: A branch is a 41-byte file under `.git/refs/heads/` holding one commit hash — creating a branch writes that file (O(1)). HEAD is normally a symbolic ref to the current branch; committing appends a commit and advances that branch pointer. A **detached HEAD** is when HEAD points *directly* at a commit instead of a branch (e.g., after `git checkout <sha>`). Commits made there aren't tracked by any branch, so they're easy to lose — the fix is to attach a branch (`git switch -c name`) before leaving, or recover via reflog afterward.

Follow-ups:

- *When do you intentionally detach?* → To inspect/build an old commit, or in CI checking out a specific SHA.
- *Recover commits made while detached?* → `git reflog` → `git switch -c rescue <sha>`.

19.11 Q11: Compare monorepo and polyrepo. Which would you choose?

Model answer: A monorepo holds everything in one repo, enabling **atomic cross-project changes** (change a library and all callers in one reviewed commit) and uniform tooling/CI, with "live at HEAD" dependencies — but it needs serious build/VCS tooling (Bazel/Buck, sparse-checkout, virtual filesystems) to scale and has a larger blast radius. A polyrepo isolates each service: small fast repos, independent deploys, fine-grained access — at the cost of cross-repo dependency/version coordination and tooling drift. I'd choose based on org structure (Conway's Law) and tooling maturity: large, tightly-coupled codebases with platform teams favor monorepo (Google/Meta); autonomous service teams with independent cadence favor polyrepo (Amazon).

Follow-ups:

- *Biggest monorepo pain?* → Clone/build scale → solved with partial clone, sparse-checkout, VFS, remote build cache.
- *Biggest polyrepo pain?* → Diamond dependencies and coordinating a breaking change across many repos.

19.12 Q12: Which branching workflow, and why?

Model answer: It depends on release cadence and CI maturity. For a continuously-deployed web service with strong CI, **trunk-based development with feature flags** (or GitHub Flow) — everyone commits small changes to `main` many times a day behind flags, so integration

is continuous and you avoid long-lived divergence and merge hell; flags decouple deploy from release. For versioned/installed software with release trains (mobile, libraries), **Git-flow** — `main`/`develop` plus `release/*` and `hotfix/*` branches give controlled stabilization. The FAANG-scale default is trunk-based with flags.

Follow-ups:

- *Why does trunk-based scale to thousands of engineers?* → Tiny, continuous merges keep everyone near HEAD → trivial conflicts; no big-bang integration.
- *How do you ship unfinished features on trunk?* → Feature flags (dark launch), plus tests guarding flagged paths.

20 Senior-Level Discussion Points

- **Deploy vs release decoupling.** Feature flags let you *deploy* code continuously while *releasing* features independently — the foundation of trunk-based development and progressive rollouts (canary, ring deployments). Mention flag-debt cleanup as the cost.
- **Commit hygiene as an engineering practice.** Atomic, single-purpose, well-described commits aren't vanity — they make `bisect` precise, `revert` safe, `blame` meaningful, and reviews fast. "Each commit should compile and pass tests" is a real bar at top shops.
- **Conventional Commits + SemVer + automated releases.** `feat:`, `fix:`, `feat!:`/`BREAKING CHANGE:` map mechanically to **Semantic Versioning** (MAJOR.MINOR.PATCH): `fix`→`patch`, `feat`→`minor`, breaking→major. Tools (semantic-release, changesets) auto-bump versions and generate changelogs from commit messages. Shows you think about release automation, not just code.
- **Force-push safety culture.** `--force-with-lease` over `--force`; branch protection (no force-push to main, required PRs, required green CI, required reviews, signed commits) — these are guardrails that prevent the worst disasters at scale.
- **CI as the real gate, hooks as fast feedback.** Local hooks are bypassable (`--no-verify`) and not shared; server-side checks (CI, `pre-receive`) are the enforcement boundary. Articulating this distinction is a maturity signal.
- **Stacked diffs / stacked PRs.** Breaking a big change into a *stack* of dependent small PRs (Graphite, Phabricator, Sapling) keeps each review small while preserving logical sequence — common at Meta and increasingly elsewhere. Rebasing the stack on `main` is routine.
- **Merge queues.** At high commit volume, "green on my branch" isn't enough because main moved; **merge queues** (GitHub merge queue, Bors, Zuul) re-test each PR against the *latest* main in order before merging, preventing semantic conflicts and broken-main races.
- **Supply-chain provenance.** Signed commits/tags (GPG/SSH/Sigstore-gitsign), protected tags, and reproducible builds tie artifacts to source — increasingly mandatory (SLSA framework).
- **Monorepo build economics.** The real cost of a monorepo isn't Git, it's *builds* — content-addressed, hermetic build systems with remote caching (Bazel/Buck) are what make "don't rebuild the world" possible.

21 Typical Mistakes Candidates Make

- **Treating Git as memorized incantations.** Not knowing *why* `reset --hard` is dangerous or what a branch physically is. Interviewers probe the model, not flags.
- **Confusing `reset` and `revert`** — proposing `reset` + force-push to undo something on a *shared* branch. That breaks teammates; `revert` is the safe public undo.
- **Not knowing what `--hard` destroys** — claiming reflog can recover *uncommitted* edits wiped

by `reset --hard`/`clean`. It can't; those were never objects.
- **Saying rebase and merge are interchangeable.** They differ in topology and whether commits are rewritten — and rebasing shared history is a cardinal sin.
- **Forgetting the Golden Rule** or being unable to explain *why* rebasing pushed commits breaks collaborators (the new-hash divergence).
- **Thinking deleting a file in a new commit removes a leaked secret.** It's still in history; you must rewrite history *and* rotate the credential.
- **Using `git push --force`** instead of `--force-with-lease`, and not mentioning branch protection.
- **`git add .` everything, vague messages** ("fixes", "stuff") — no atomicity, useless for blame/bisect/review.
- **Resolving a conflict by deleting one side blindly** or leaving stray `<<<<`/`>>>>` markers in committed code.
- **Not knowing `reflog` exists** — panicking and re-cloning when recovery was one command away.
- **Overusing cherry-pick** as a merge substitute, creating duplicate commits and later conflicts.
- **Choosing a workflow dogmatically** rather than tying it to release cadence, team size, and CI maturity.

22 How This Connects to Other Topics

- **Operating Systems:** Git is a content-addressable *filesystem*; blobs/trees mirror inodes/directories, and packfiles are storage compaction. `.git/index` is a cache like a page table. File-watching tools (and VFS-for-Git) hook OS filesystem layers.
- **Hashing & Data Structures:** Git is a **Merkle DAG** — the same hash-chained structure as blockchains and Merkle trees in distributed systems. SHA-1/SHA-256 content addressing = consistent hashing's cousin for integrity.
- **System Design / CI-CD:** Commits are immutable build inputs; pipelines key off SHAs. Merge queues, canary/blue-green deploys, and rollback strategies all ride on Git semantics. Feature flags connect to release engineering.
- **Distributed Systems:** Git's fetch/push is replication with eventual consistency between clones; divergence + merge = conflict resolution; force-push = a (dangerous) "last writer wins."
- **Low-Level Design / Code Review:** Clean class design pairs with clean commit/PR design — small, single-responsibility units in both code and history.
- **Security:** Signed commits/tags = authenticity; secret scanning + history scrubbing = supply-chain hygiene; branch protection = access control.

23 FAANG Interview Tips

- **Lead with the model, not the command.** "A branch is just a pointer; commits are immutable snapshots hash-chained to their parents." This framing instantly reads as senior.
- **For any 'undo' question, state safety first:** local vs shared. Reset/rebase for local; revert/`--force-with-lease` for shared. Always mention `reflog` as the safety net.
- **Draw the graph.** Merge-base diagrams, before/after rebase, the three areas — a quick ASCII sketch communicates more than paragraphs.
- **Tie workflows to constraints.** Don't say "Gitflow is best." Say "for continuous deploy with strong CI, trunk-based + flags; for versioned releases, Gitflow."

- **Show disaster composure.** "First I'd `git reflog`, then choose the least destructive fix." Calm + correct beats fast + reckless.
- **Mention guardrails unprompted.** Branch protection, required CI, `--force-with-lease`, signed tags, secret scanning — shows you think about team safety at scale.
- **Use the precise vocabulary:** merge base, fast-forward, detached HEAD, dangling commit, content addressing, Merkle DAG, packfile, `ort` strategy, sparse-checkout.
- **Connect to scale.** Monorepo build tooling, VFS, merge queues, stacked diffs — signals you've thought beyond a 5-person repo.

24 Revision Cheat Sheet

Mental model

- Git = content-addressed key-value store of 4 objects: **blob** (file bytes), **tree** (dir), **commit** (tree+parent+meta), **tag** (named pointer).
- Branch = 41-byte file holding a hash. HEAD = "where am I." Commits are immutable; "editing history" = new hashes + moved pointer.
- Commit hash = SHA-1 of `"commit <len>\0"` + text, and the text **includes parent+tree hashes** → tamper-evident Merkle DAG.

Three areas

- Working dir → `add` → Index (staging) → `commit` → Repo. `restore`/`reset` move left. `add -p` for partial staging.

Reset modes (what each touches)

Mode	HEAD	Index	Working dir
`--soft`			(changes staged)
`--mixed`			(changes unstaged)
`--hard`			(changes DESTROYED)

Undo decision

- Local, unpushed → `reset` (soft/mixed safe; hard destroys uncommitted).
- Pushed/shared → `revert` (inverse commit; never rewrite shared history).
- Lost commits → `git reflog` → `reset --hard <sha>` / `switch -c rescue <sha>`.

Merge vs Rebase

- Merge = preserve topology, merge commit, original hashes. Rebase = linear, **new hashes**, replay. **Golden Rule: never rebase pushed/shared commits.** `pull --rebase` for clean local history.

Conflict

- 3-way: base + ours + theirs. Conflict = both changed same region. Resolve, remove markers, `add`, `--continue`. `--ours`/`--theirs` per file. `zdiff3` + `rerere` make life easier.

Power tools

- `reflog` (recover anything), `bisect` (O(log n) regression hunt, `bisect run`), `cherry-pick` (backport one commit), `stash` (shelve WIP), `worktree` (parallel branches), `blame`/`log -S`/`log -L`.

Workflows

- Gitflow (versioned releases), GitHub Flow (continuous deploy), GitLab Flow (env branches), **Trunk-Based + feature flags** (FAANG scale; continuous tiny merges avoid merge hell).

Monorepo vs Polyrepo

- Mono: atomic cross-project change + uniform tooling; needs Bazel/Buck/VFS/sparse-checkout (Google/Meta). Poly: isolation + independent deploys; dependency/version coordination cost (Amazon).

Disaster one-liners

- Committed to main: `git branch feature && git reset --hard origin/main`.
- Undo pushed commit: `git revert <sha>`.
- Lost via hard reset: `git reflog` → `git reset --hard <sha>` (or `ORIG_HEAD`).
- Clobbered teammate: restore via reflog; prevent with `--force-with-lease` + branch protection.
- Leaked secret: **rotate first**, `git filter-repo --path <f> --invert-paths` / BFG, force-push, re-clone.
- Detached HEAD: `git switch -c rescue` (before leaving) or reflog after.

Hygiene

- Atomic, single-purpose commits that compile/pass tests. Conventional Commits (`feat`/`fix`/`feat!`) → SemVer (minor/patch/major) → automated changelogs. Small PRs. `--force-with-lease`, not `--force`. Sign release tags.

Golden sentences for the room

1. ”A branch is just a pointer; Git is a Merkle DAG of immutable snapshots.”
2. ”Reset rewinds (local only); revert records an inverse (safe for shared).”
3. ”Rebase locally for clean history; never rebase what you've pushed.”
4. ”Don't panic — `git reflog` first, then the least destructive fix.”